Data Science Workshop

Weather Forecasting

By: Elior Segal

In this Data Science project, I experimented with a weather forecasting task.

My main objective was to predict whether it will rain tomorrow.

Additionally, I had a secondary objective to impute sunshine and evaporation values, which assists in predicting rain more accurately.

Dataset

https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package/data

I used the **Rain in Australia** dataset from Kaggle.

The dataset comprises about 10 years of daily weather observations from numerous locations (weather stations) across Australia.

Additionally, it provides a target variable: **RainTomorrow**, which answers the crucial question: will it rain the next day? (Yes or No).

The observations were gathered from an Australian weather governmental site: https://www.bom.gov.au/climate/data/.

Tasks

My primary objective and the dataset's original goal is predicting whether it will rain tomorrow.

However, a secondary objective was added to impute the sunshine and evaporation columns, as they contain many null values.

Motivation

Weather forecast is important for everyday life, for example knowing whether an umbrella is needed.

Moreover, it is important for safety in case of an emergency or even for daily transportation of people or supplies.

And to predict the weather accurately I need to impute Sunshine, which is a key factor for predicting tomorrow's rain:

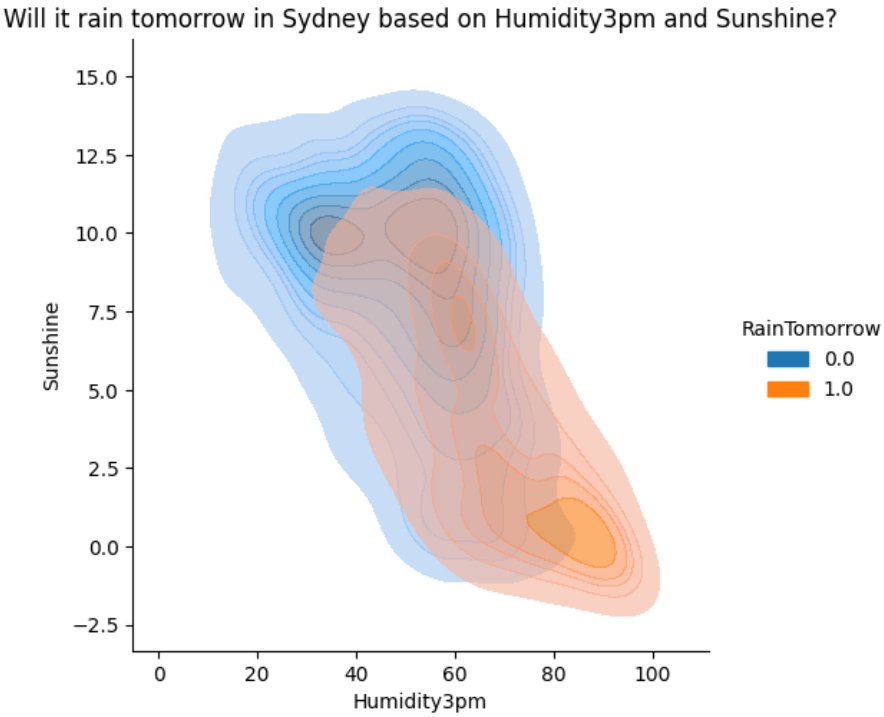


Figure 1; We can see that using only the 2 most important features – Sunshine and Humidity at 3pm we can get a non-trivial prediction of the rain tomorrow.

Features

Dataset size: 145460 rows.

Date - The date of observation (1/11/2007 - 25/6/2017).

Location - The common name of the location of the weather station.

MinTemp - The minimum temperature in degrees celsius.

MaxTemp - The maximum temperature in degrees celsius.

Rainfall - The amount of rainfall recorded for the day in mm.

Evaporation - The so-called Class A pan evaporation (mm) in the 24 hours to 9am.

Sunshine - The number of hours of bright sunshine in the day.

WindGustDir - The direction of the strongest wind gust in the 24 hours to midnight.

WindGustSpeed - The speed (km/h) of the strongest wind gust in the 24 hours to midnight.

WindDir9am - Direction of the wind at 9am.

WindDir3pm - Direction of the wind at 3pm.

WindSpeed9am - Wind speed (km/hr) averaged over 10 minutes prior to 9am.

WindSpeed3pm - Wind speed (km/hr) averaged over 10 minutes prior to 3pm.

Humidity9am - Humidity (percent) at 9am.

Humidity3pm - Humidity (percent) at 3pm.

Pressure9am - Atmospheric pressure (hpa) reduced to mean sea level at 9am.

Pressure3pm - Atmospheric pressure (hpa) reduced to mean sea level at 3pm.

Cloud9am - Fraction of sky obscured by cloud at 9am. This is measured in "oktas", which are a unit of eigths. It records how many eigths of the sky are obscured by cloud. A 0 measure indicates completely clear sky whilst an 8 indicates that it is completely overcast.

Cloud3pm - Fraction of sky obscured by cloud (in "oktas": eighths) at 3pm. See Cload9am for a description of the values.

Temp9am - Temperature (degrees C) at 9am.

Temp3pm - Temperature (degrees C) at 3pm.

RainToday - Boolean: Yes if precipitation (mm) in the 24 hours to 9am exceeds 1mm, otherwise No.

RainTomorrow - Boolean: Yes if precipitation (mm) in tomorrows 24 hours to 9am exceeds 1mm, otherwise No.

Notes:

- Date and location are some kind of metadata (still will be used as features).
- RainTomorrow is the label and won't be used as a feature.
- I have also enriched the data which I will expand on later.

Feasibility Analysis

As shown in Figure 1, by using the 2 most important features we can achieve non-trivial predictions of tomorrow's rain.

Additionally, for the sunshine prediction, just by using the Humidity at 3pm and the Temperature at 3pm we can predict relatively accurately the sunshine:

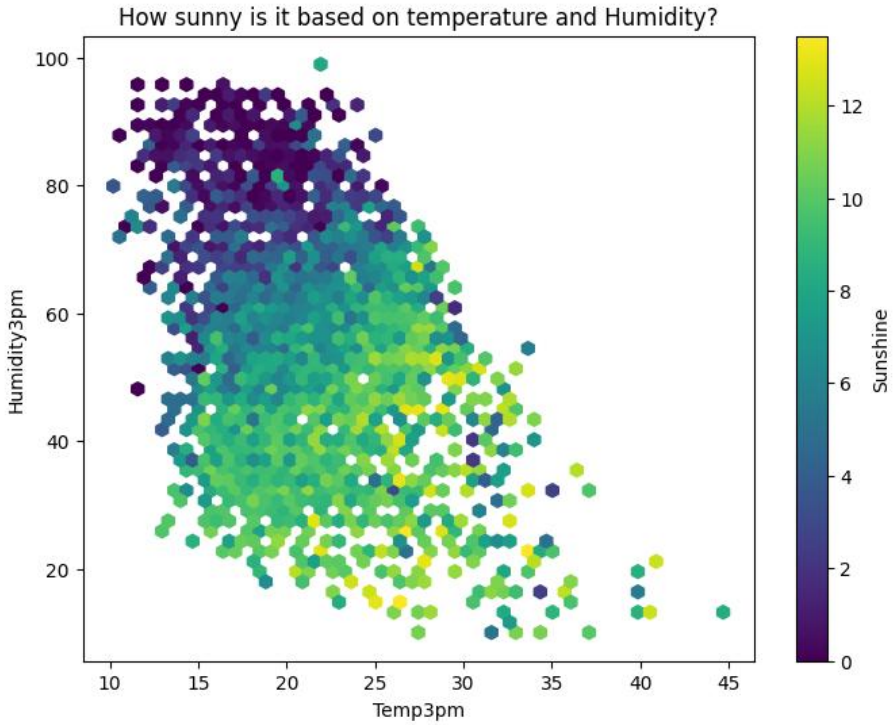


Figure 2 Daily sunshine as a function of Humidity at 3pm and the temperature at 3pm

Null Amounts

One of the challenges in the project was the high number of missing values in some columns:

Column	Null %		Column	Null %
Date	0.00		**WindSpeed3pm**	2.11
Location	0.00		**Humidity9am**	1.82
MinTemp	1.02		**Humidity3pm**	3.10
MaxTemp	0.87		**Pressure9am**	10.36

Column	Null %		Column	Null %
Rainfall	2.24		**Pressure3pm**	10.33
Evaporation	43.17		**Cloud9am**	38.42
Sunshine	48.01		**Cloud3pm**	40.81
WindGustDir	7.10		**Temp9am**	1.21
WindGustSpeed	7.06		**Temp3pm**	2.48
WindDir9am	7.26		**RainToday**	2.24
WindDir3pm	2.91		**RainTomorrow**	2.25
WindSpeed9am	1.21			

I handled them in 3 different ways:

1. Using time interpolation if the feature supports it.
2. Using Iterative imputers.
3. Using a dedicated model.

Feature Engineering

Encodings

Wind Direction

Wind directions were provided as a “compass direction “. For example, “N”, “S”, “NE”, “SW”.

To keep close directions numerically similar, instead of using one-hot encoding I encoded the direction as a point on a unit circle representing the direction:

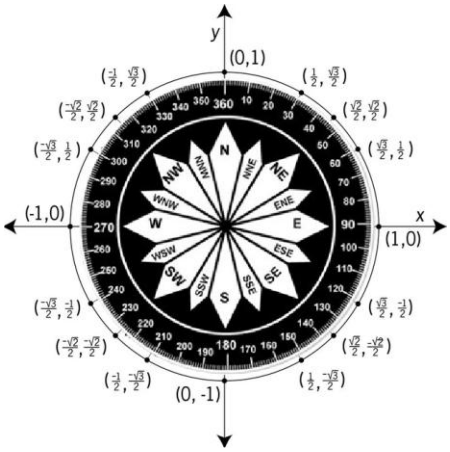
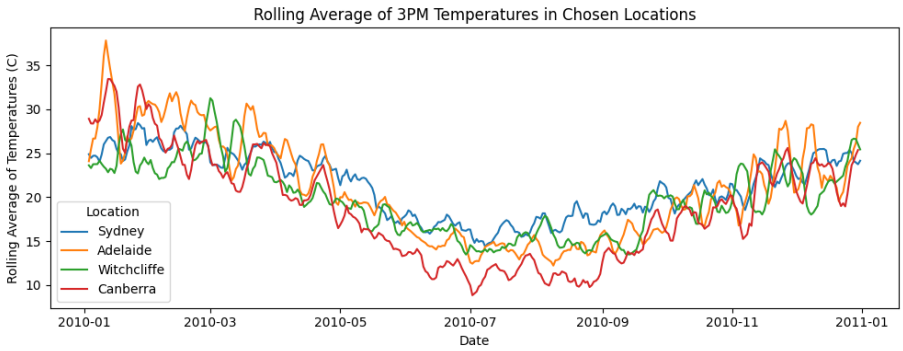


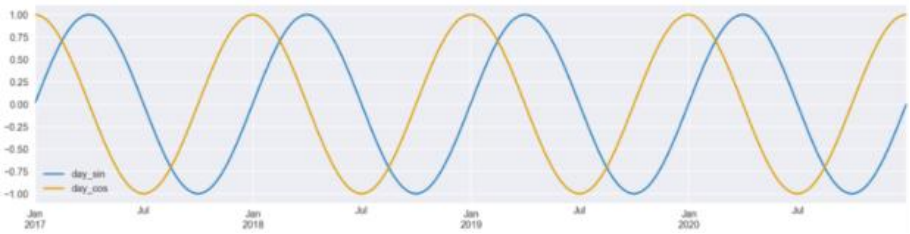
Figure 3 The numerical representation given to each wind direction.

Time (Date)

In a weather data obviously, there is a strong yearly seasonality (seasons...).



So, I encoded the day of the year as the corresponding values of sine and cosine functions (this way, Jan 1st and Dec 31st are numerically similar).



Location

There are 49 different locations provided by the weather station's name.

I encoded them using longitude and latitude values (rather than using one-hot encoding):

Why?

Because close location should have similar climates – so we provide the model with this knowledge rather than letting it behave similarly in close locations on its own.

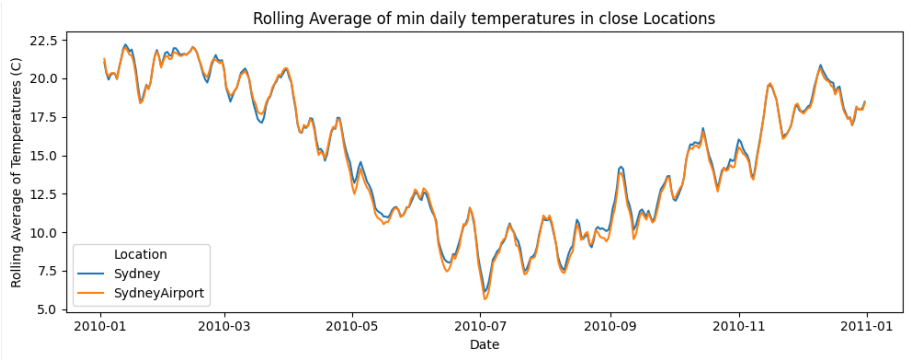


Figure 4 Minimal daily temperature in Sydney and Sydney's airport, obviously incredibly similar

Commented [DN1]: This graph wasn't in the presentations

Additionally, this makes the feature space more compact instead of providing the model with 49 additional features, most of which are redundant or unimportant.

How?

To enrich the data with the stations' locations I gathered from the weather site a table of all their stations and looked up the correct locations based on names and time of operation (if a station was active between 1978-1986 it isn't the station corresponding to our dataset).

Additionally, I normalized the locations' latitude and longitude in correspondence to an arbitrary center point of Australia (And also provided the squared value of the normalized coordinates).

I did it because I wanted the model to treat Australia like a normalized circle rather than have big values/values that correspond to edge case's locations (for example, a side island like Norfolk Island).

Simple Null Handling

Temporal Features

I imputed time-continuous features using pandas time interpolation (meaning to impute a value we take an average weighted based on the time of its neighboring rows – sorted by time).

To decide which features are imputable using this technique, I plotted them throughout the year in a selected location and saw how continuous it is:

Note, the gap in the provided figures is due to missing rows rather than null values.

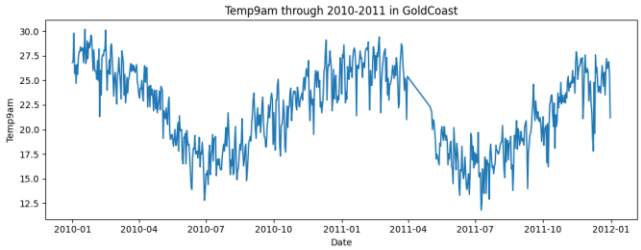


Figure 5 Example of a time continuous feature.

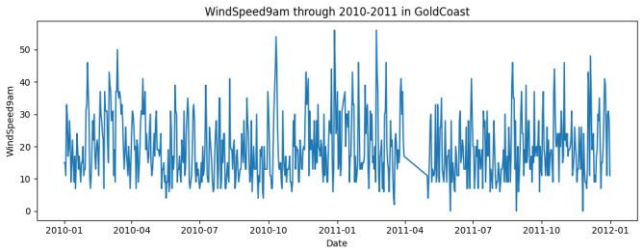


Figure 6 Example of a time discontinuous feature.

After imputing them using time interpolation, some features had an incredibly low null percentage, so I dropped their rows (because they don't matter):

MinTemp (0.0014%), MaxTemp (0.0007%), Temp9am (0.0021%), Temp3pm (0.0021%).

Additionally, I decided to drop columns with null values in "RainToday" because it also had a low null (<1%) percentage and it was the same rows as the rows where Rainfall is null.

Iterative Imputer

Scikit-learn's Iterative Imputer is a sophisticated tool used to handle missing data by modeling each feature with missing values as a function of the other features.

Unlike simple imputation (which just fills in the mean or median), the Iterative Imputer treats missing value estimation as a regression problem.

Iterative Imputer Evaluation

Even if I use the iterative imputer, how can I know whether it does a good job?
And if it does do a good job, why not use it to impute the temporal features as well?

Iterative imputer vs time interpolation

Methodology

I took the complete rows from the training data and randomly inserted some null values in the temporal features.

Then I imputed them using the time interpolation approach and using the iterative imputer (I used the extra trees regressor as we'll later see it's a strong regressor for the iterative imputer and a simpler model - KNeighborsRegressor).

I measured the RMSE of each approach for each column to see which one is better.

Results

Extra trees result:

Column	Time-Interp RMSE	Iterative RMSE
MinTemp	2.52	4.83
MaxTemp	2.85	5.67
Pressure9am	3.86	6.00
Pressure3pm	3.63	6.08
Temp9am	2.52	4.88
Temp3pm	2.98	5.39

KNeighborsRegressor result:

Column	Time-Interp RMSE	Iterative RMSE
MinTemp	2.49	3.29
MaxTemp	2.96	3.23
Pressure9am	3.84	4.93
Pressure3pm	3.72	4.96
Temp9am	2.44	2.95
Temp3pm	3.14	3.12

As we can see, the time interpolation approach was a lot better.

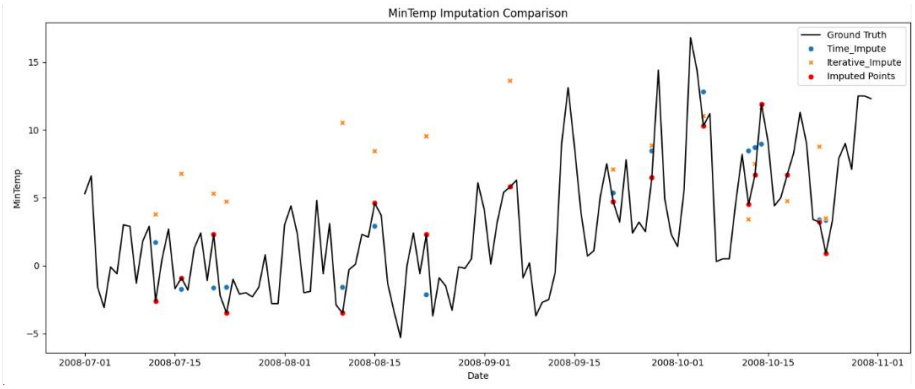


Figure 7 Extra trees iterative imputation vs time imputation

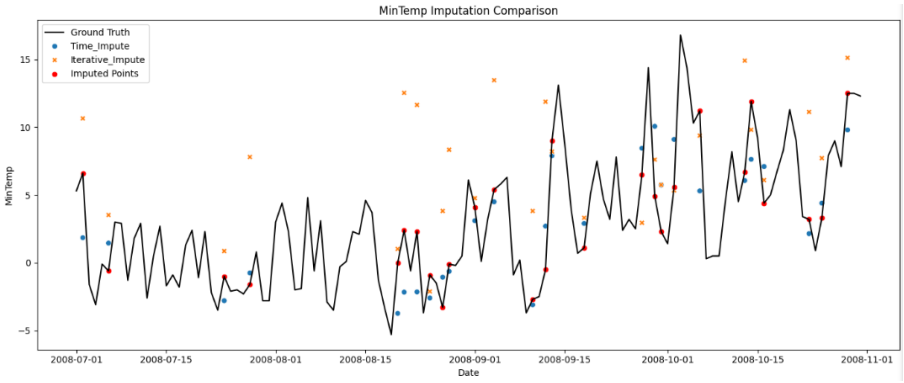


Figure 8 K-Neighbors iterative imputation vs time imputation

Commented [DN2]: These graphs weren't in the presentations

Different Iterative Imputer's estimators

We want to know which estimator to use to impute most accurately the other features.

Methodology

I took the complete rows and randomly split them into train and validation sets.

Then I randomly inserted some null values based on the null percentage of each column in the original data.

I scaled the dataset to help some estimators and have been able to compare the accuracy between different columns (with the same scale).

I trained each iterative imputer (with different estimators) on the train set and checked its MSE and R2 scores on the validation set.

I tested the following estimators:

BayesianRidge (iterative imputer's default)

ExtraTrees

Nystroem + Ridge

K-Neighbors

K-Neighbors-large (more neighbors)

Random Forest

Results:

Estimator	MSE_mean	R2_mean	Time (s)
ExtraTrees	0.4506	0.5517	143.94
K-Neighbors-large	0.4718	0.5307	62.56
BayesianRidge	0.4816	0.5209	14.24
Random Forest	0.5077	0.4949	214.60
K-Neighbors	0.5132	0.4895	61.25
Nystroem + Ridge	0.7037	0.3000	8.50

I also did a per column check:

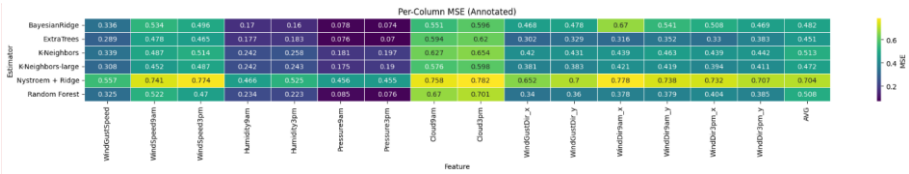


Figure 9 Iterative imputer MSE with different estimators

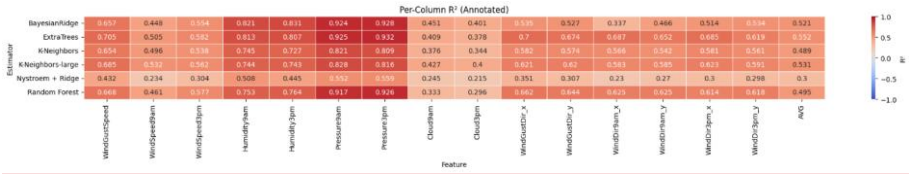


Figure 10 Iterative imputer R^2 with different estimators

Commented [DN3]: These graphs weren't in the presentations

We can see that ExtraTrees is the best model, unfortunately it takes too much memory.

So instead, I chose to use the Random forest and K-Neighbors-large models (dependent on the column) – each with its strength.

Commented [DN4]: In the presentation I used only random forest

I also plotted the estimators' prediction vs the real value of each column.
(It couldn't be a line plot like figure 6 because the data are random rows rather than a continuous portion)

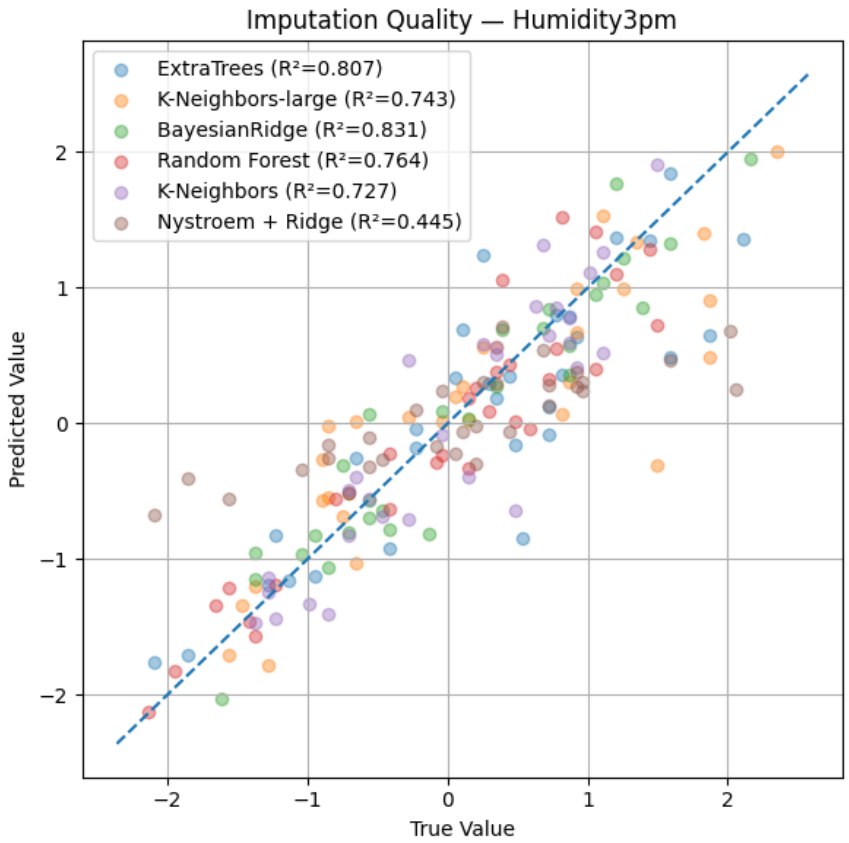


Figure 11 Iterative imputer with different estimators accuracy to real value prediction.

Commented [DN5]: This graph wasn't in the presentations

Sunshine Imputation

Sunshine has an extreme number of missing values; the main reason is that some locations in the dataset don't have any sunshine information:

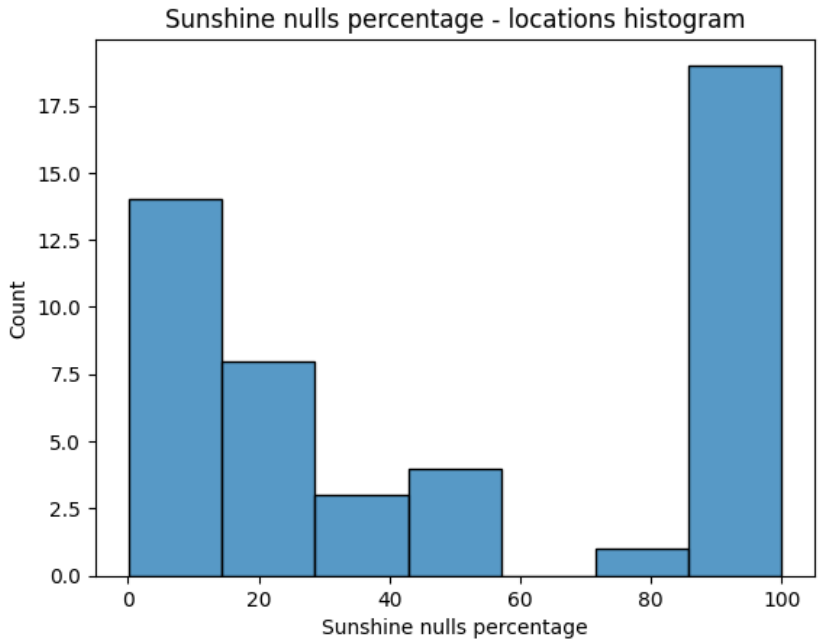


Figure 12 Location's missing sunshine percentage distribution

Methodology

1. Enrich the data with correlated information.
2. Fit models on locations with low null amounts.
3. Evaluate the models on locations with medium null amounts.
4. Fill all the missing null values using the best model.

Solar Exposure

Originally, I wanted to just gather sunshine information from the original site. Unfortunately, it seemed to be behind a paywall. Nonetheless, I found out about another feature – **solar exposure** which seem highly correlated with sunshine, so I chose to enrich my data by collecting it and use it to impute the sunshine.

Global solar exposure

Global solar exposure is the total amount of solar energy falling on a horizontal surface. The daily global solar exposure is the total solar energy for a day. Typical values for daily global solar exposure range from 1 to 35 MJ/m^2 (megajoules per square metre). The values are usually highest in clear sun conditions during the summer, and lowest during winter or very cloudy days.

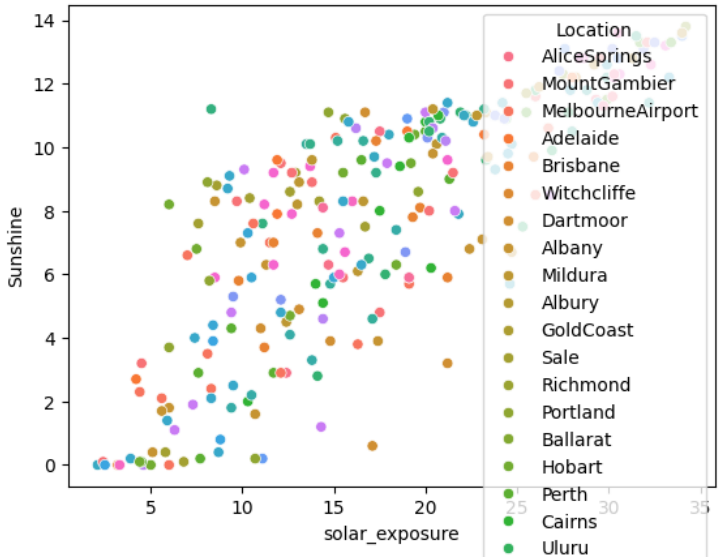


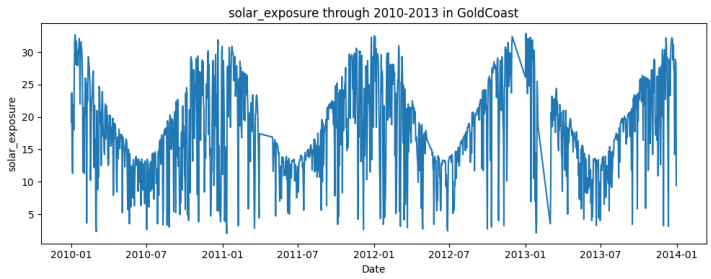
Figure 13 Solar exposure correlation to sunshine across different locations

To collect the solar exposure from the correct stations I again used the same stations information which I used for their location.

Then I scraped the corresponding page and gathered it together to a single file.

Notes:

- Solar exposure also had some null values which I imputed using the month's median:



- In some locations there was no Solar exposure information available at all.

Modeling

I trained on locations with less than 10% of nulls and evaluated on locations with 10%-20% of nulls.

On locations with solar exposure:

Model	MSE	R2
XGBoost	3.6604	0.7504
Gradient Boosting	3.6984	0.7478
Random Forest	3.8148	0.7399
Lasso	4.0069	0.7268
Ridge	4.1164	0.7193
Linear Regression	4.1166	0.7193
MLP (Neural Network)	4.2869	0.7077

As noted, I had to provide a solution for locations without solar exposure as well, so I trained also without providing the solar exposure:

Model	MSE	R2
XGBoost	4.2068	0.7085
Gradient Boosting	4.3982	0.6952
Random Forest	4.4126	0.6942
Ridge	5.0676	0.6488
Linear Regression	5.0676	0.6488

Model	MSE	R2
Lasso	5.1885	0.6405
MLP	15.7630	-0.0923

Interestingly, the solar exposure played a big role in computing sunshine accurately:

1. The most important feature of the best model is solar exposure with 46.1% importance.
2. We can clearly see a drastic decrease in performance when solar exposure isn't provided.

At the end I used the XGBoost models with or without the solar exposure provided correspondingly.

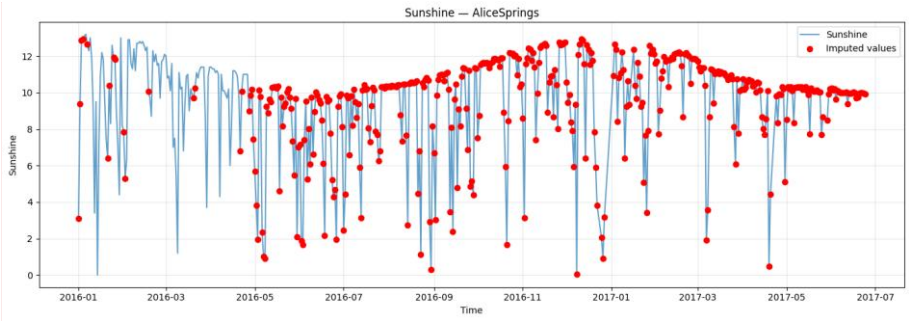


Figure 14 Sunshine imputation in location with sunshine provided – imputation clearly follows the trend

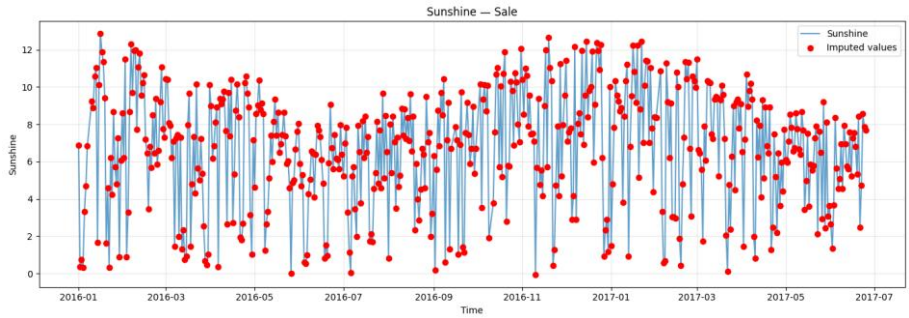


Figure 15 Sunshine imputation in location without sunshine provided – imputation seems random

Commented [DN6]: These graphs weren't in the presentations

Evaporation Imputation

Similarly to sunshine, evaporation has extreme number of null values because some locations don't have any evaporation information at all.

Therefore I used a similar methodology:

Methodology

1. Fit models on locations with low null amounts.
2. Evaluate the models on locations with medium null amounts.
3. Fill all the missing null values using the best model.

Unfortunately, it went terribly:

Model	MSE	R2
Lasso	18.8709	-0.0176
Ridge	20.3728	-0.0986
Linear Regression	20.3741	-0.0986
Gradient Boosting	21.4938	-0.1590
XGBoost	21.9023	-0.1810
Random Forest	25.8164	-0.3921
MLP	28.6044	-0.5424

The reason is that evaporation is less correlated with other features, additionally on small scales it behaves randomly and it's a lot better to catch its overall trend:

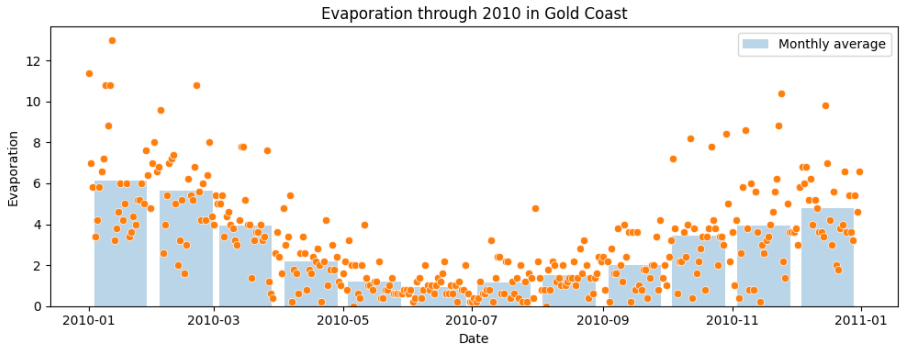


Figure 16 Evaporation follows a monthly trend

Indeed, when using the monthly average for each location we get better results from all the models:

MSE: 13.41, R2: 0.276

Even if we use a lasso model with only the location and time provided, we get better results from the models above:

MSE: 18.83, R2: -0.015

Grids

Again, similarly to sunshine imputation I tried to enrich my data using the weather site.

The only evaporation information that was available was monthly and yearly averages across locations:

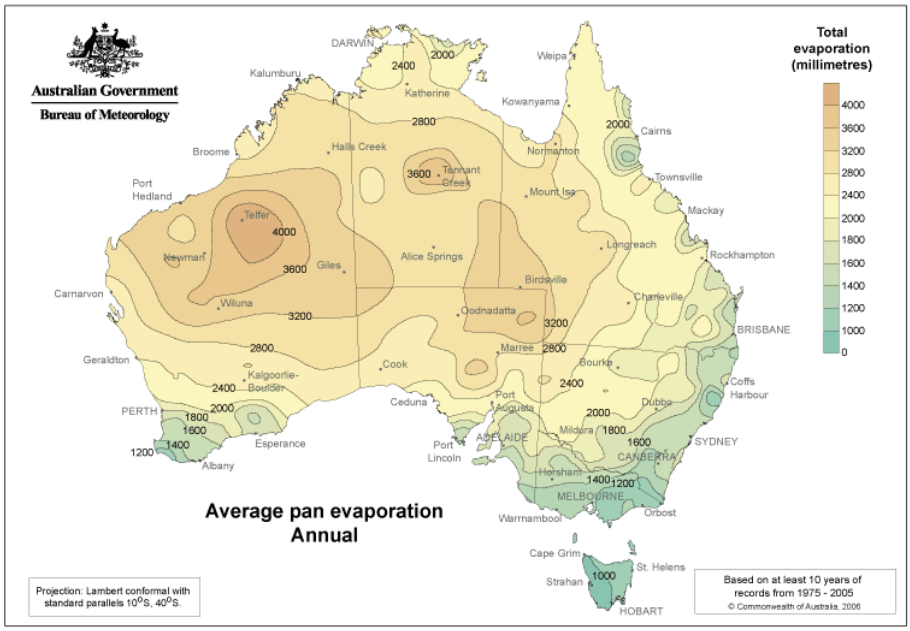


Figure 17 Average evaporation across different locations

On a first look that seems exactly what we need – the average evaporation.

Unfortunately, due to global warming, these averages are outdated as evaporation rates have increased significantly:

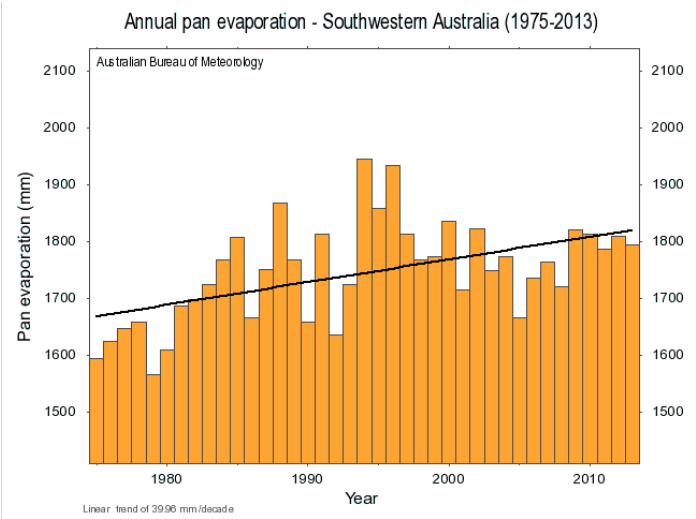


Figure 18 Evaporation increases over the years.

Commented [DN7]: This graph wasn't in the presentation.
Also, its from the interenet - my data doesn't span over so many years

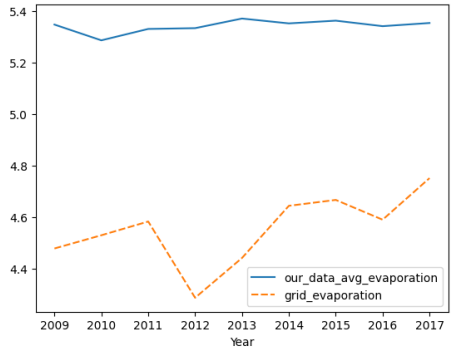


Figure 19 Our data evaporation vs average grid evaporation (our clearly larger)

Also, when plotting the evaporation, inexplicably, it seemed shifted:

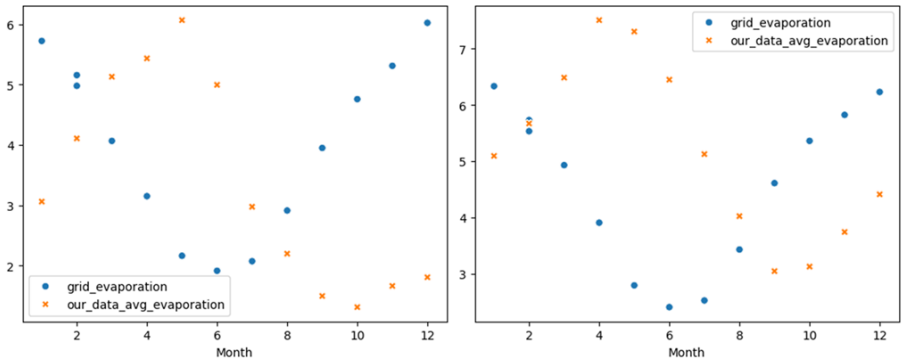


Figure 20 our data evaporation vs average grid evaporation (Norah Head and Coffs Harbour) (seem shifted by 4 months)

Overall, there was almost no reason to use the evaporation grids:

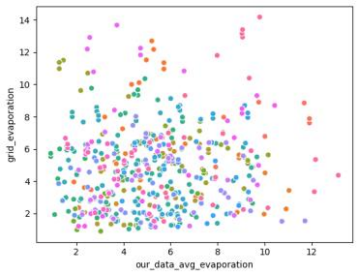


Figure 21 almost no correlation between grids evaporation and our data

Commented [DN8]: These graphs weren't in the presentation.

I imputed evaporation using the monthly average of the data and only in locations it wasn't possible to, I used the grids information.

Modeling

To have a constant train and evaluation dataset I split the dataset into train validation and test at the start and used the same split across the project.

Additionally, I split the dataset based on the years that the train is at the beginning (2007-2013), while the validation (2014-2015) and test are later (2016-2017) to model correctly the accuracy of being able to predict correctly in the future too.

Baseline

The classic saying: "The best model to predicting if there will be rain tomorrow, is whether there was rain today"

accuracy : 77%

precision : 44%

recall : 45%

f1 : 45%

confusion_matrix :

[[23296 3949]

[3874 3151]]

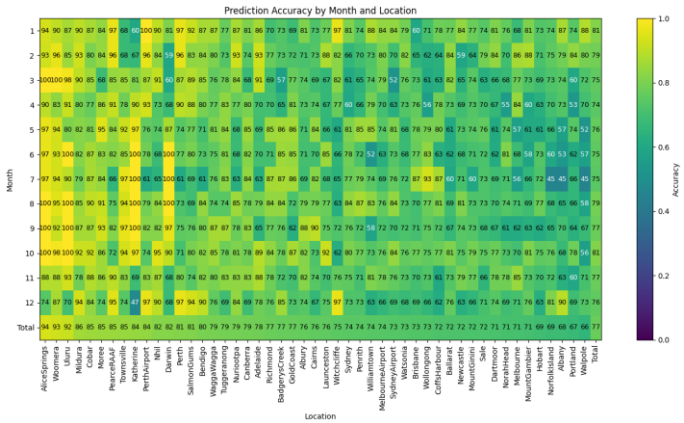


Figure 22 Baseline accuracy across months and locations

After Simple Null Handling

(Tried random forest as well, but it did a little poorer).

Using the XGBoost model we get an 86.38% accuracy and 61% f1 score.

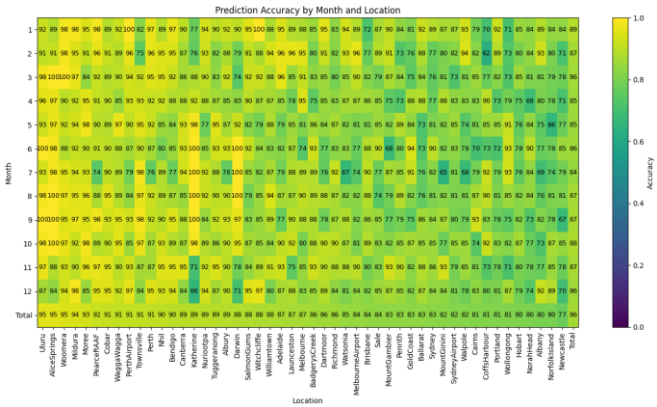
With the top feature importance as:

Humidity3pm - 0.281023

Cloud3pm - 0.132366

WindGustSpeed - 0.063947

Rainfall - 0.051682



After Imputing and Providing Sunshine

(Tried random forest as well, but it did a little poorer).

Using the XGBoost model we get an 86.74% accuracy and 63% f1 score.

With the top feature importance as:

Humidity3pm 0.207086

Sunshine 0.170441

Cloud3pm 0.060945

WindGustSpeed 0.053064

- Note as Sunshine is the second most important feature which means imputing it was important and meaningful.

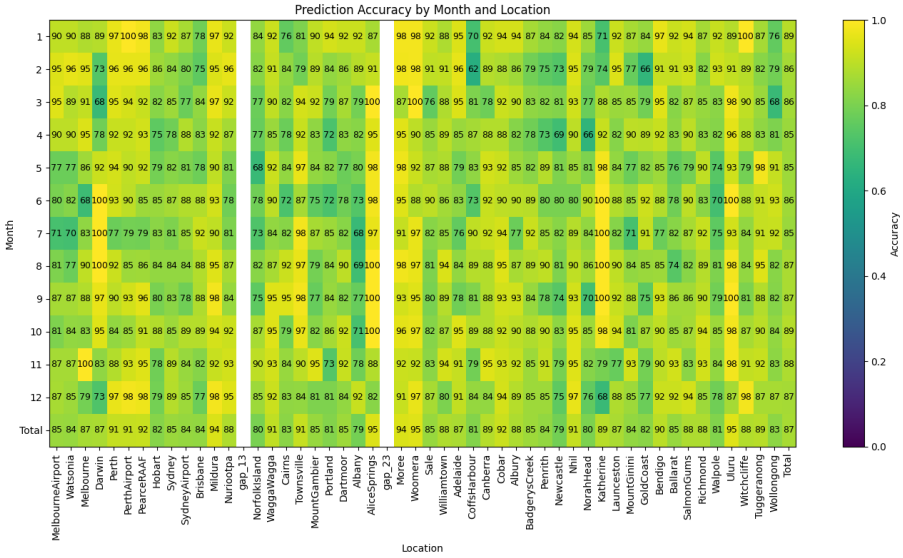


Figure 23 Accuracy per month and location grouped by train-validation-test split of sunshine locations

- Note as there isn't a large difference in performance between locations with sunshine imputed and trained which suggests that the imputation succeeded accurately.

After Imputing and Providing Evaporation

(Tried random forest and extra trees as well, but they did a little poorer).

Using the XGBoost model we get an 86.67% accuracy and 63% f1 score.

With the top feature importance as:

Humidity3pm 0.221430

Sunshine 0.163460

WindGustSpeed 0.054996

Cloud3pm 0.052547

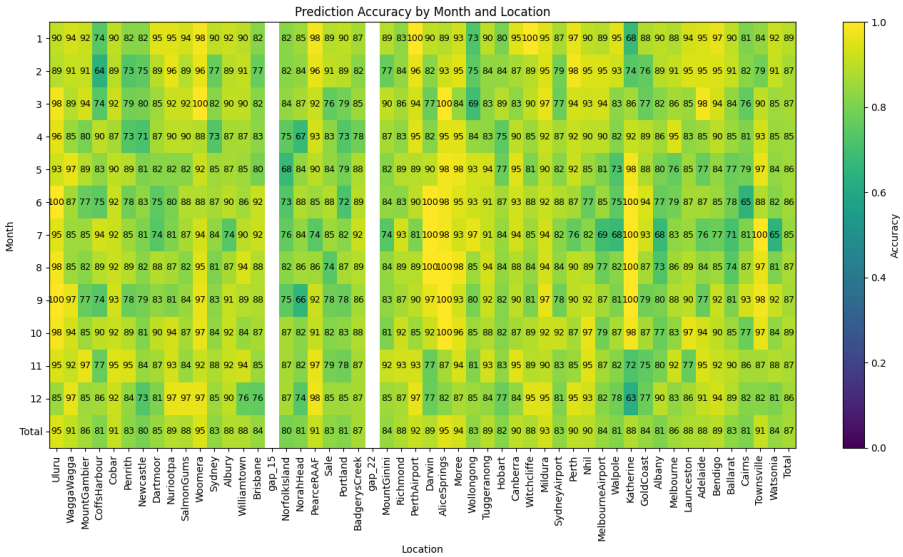


Figure 24 Accuracy per month and location grouped by train-validation-test split of evaporation locations

Temporal Locality

I wanted to check whether providing several days of features (instead of just one) would improve the model.

I did it by adding “lagged” columns of “k” previous day.

Note, some columns don’t need to be lagged/duplicated because they are constant (such as the location) or deducible (such as time).

	Date	MinTemp	MinTemp_lag1	MinTemp_lag2
137834	2017-06-17	9.0	11.6	7.6
137835	2017-06-18	8.3	9.0	11.6
137836	2017-06-19	5.5	8.3	9.0
137837	2017-06-20	5.6	5.5	8.3
137838	2017-06-21	5.8	5.6	5.5
137839	2017-06-22	7.8	5.8	5.6
137840	2017-06-23	8.0	7.8	5.8
137841	2017-06-24	11.5	8.0	7.8

Figure 25 Example of lagged feature, each row provides information from the last 2 days as well

K = 1

(Tried random forest and different XGBoost model sizes, but they did a little poorer).

Using a large XGBoost model we get an 86.71% accuracy and 62% f1 score.

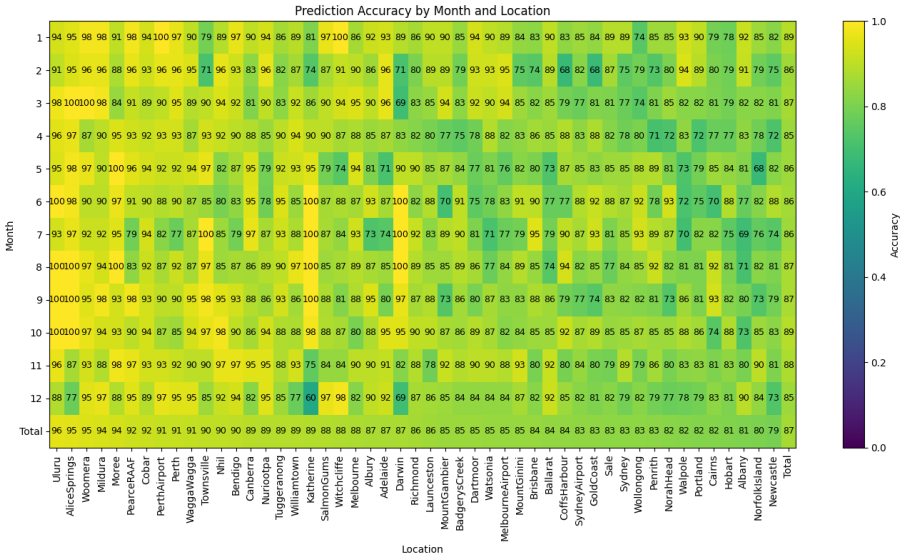
With the top feature importance as:

Humidity3pm 0.171677

Sunshine 0.087806

WindGustSpeed 0.040400

Pressure3pm 0.037472



K = 2

(Tried random forest and another XGBoost mode, but they did a little poorer).

Using a large XGBoost model we get an 86.58% accuracy and 62% f1 score.

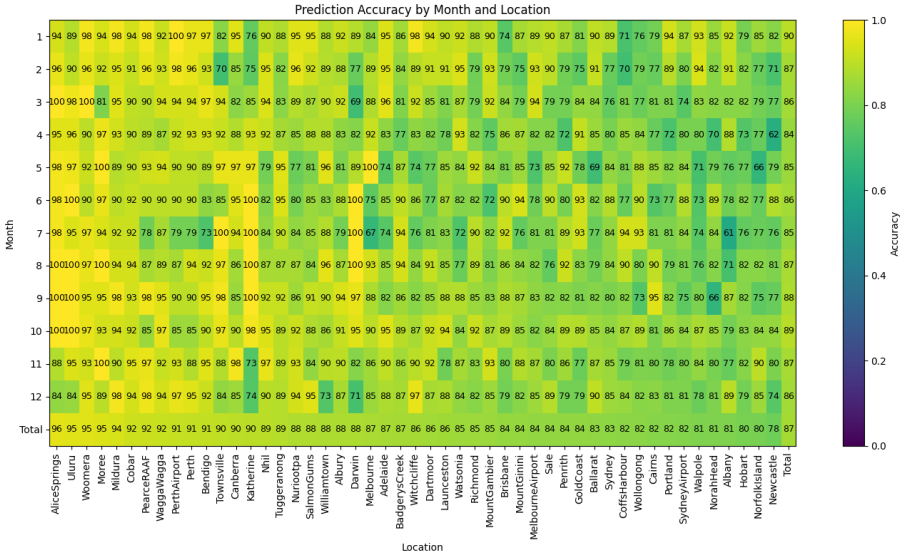
With the top feature importance as:

Humidity3pm 0.119622

Sunshine 0.089259

WindGustSpeed 0.032744

Pressure3pm 0.030290



Space Locality

Additionally, I wanted to check if providing information from nearby locations can improve the model.

But instead of defining what a nearby location is, I provided the model data from all the locations at once (hoping it will get better results).

Unfortunately, I didn't have enough data (because if 1 location has missing data on a certain date, I didn't create that row).

Consequently, the results were meaningless...

Different Architectures and Hyperparameters Tuning

I wanted to use Optuna (An open-source hyperparameter optimization framework to automate hyperparameter search) to tune my model parameters.

However, before running an Optuna study, I wanted to check what architecture to optimize. So, I trained several machine learning models on the data and compared their accuracy on the validation:

- Note, special columns are approximately half of the most important features according to the default XGBoost model – I compared using only them as features vs using all the features.

	model	features	val_score
0	XGBoost	all_features	0.869624
1	XGBoost	special_columns	0.867260
2	RandomForest	all_features	0.865713
3	RandomForest	special_columns	0.863204
4	KNN	special_columns	0.858681
5	LogisticRegression	all_features	0.857747
6	MLP	special_columns	0.855996
7	KNN	all_features	0.853662
8	LogisticRegression	special_columns	0.853137
9	MLP	all_features	0.852261

Figure 26 Comparison of different models with different input features validation accuracies

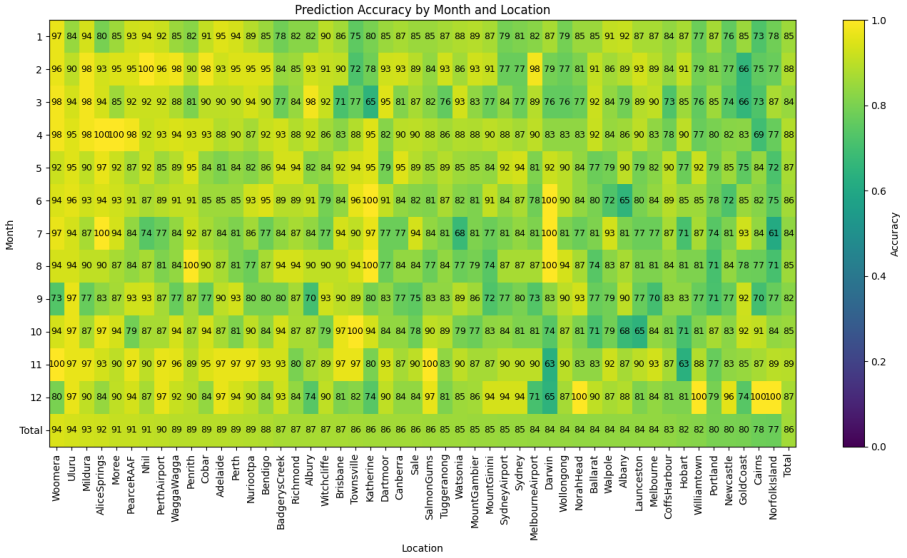
In conclusion, I hyperparameter-tuned XGBoost with all the features provided.

XGBoost Hyperparameter Tuning

I ran 100 Optuna trials.

My best validation accuracy was an 87.17% and 64% f1 score.

With test accuracy of 86.11% and 66% f1 score.



By looking at the trials' parameters, my ranges were spot on, except for max depth which should have had higher upper bound:

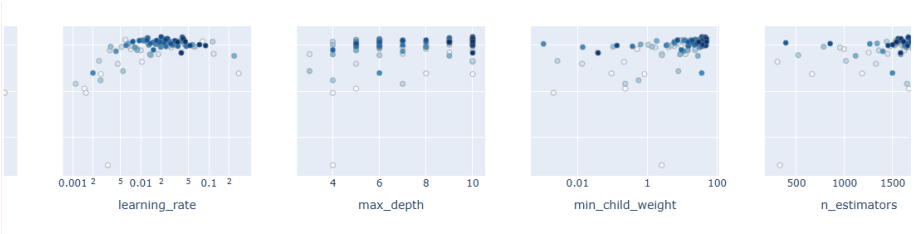


Figure 27 Trials validation accuracy as a function of the parameter

MLP Hyperparameter Tuning

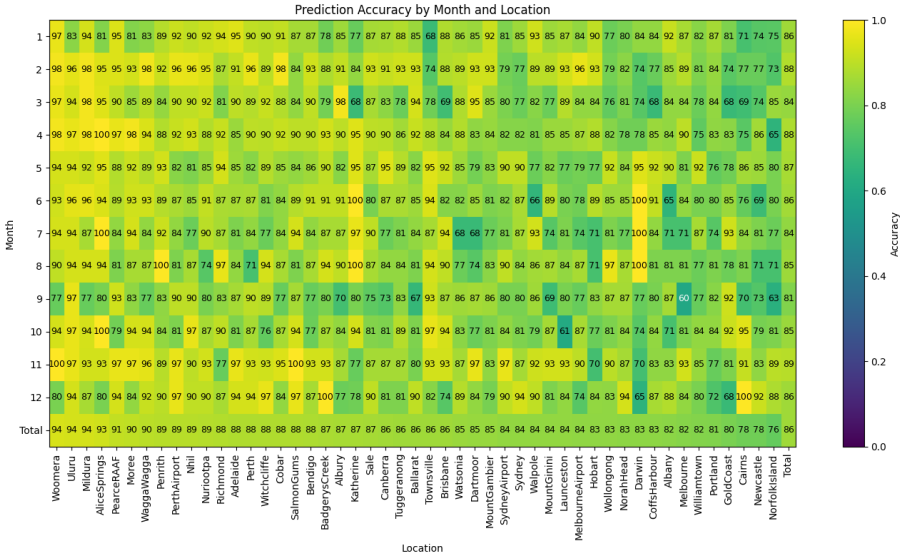
I ran 100 Optuna trials.

My best validation accuracy was an 87.02% and 63% f1 score.

With test accuracy of 85.92% and 65% f1 score.

Commented [DN9]: These graphs weren't in the presentation.

Commented [DN10]: This section wasn't in the presentation.



The best network had 3 layers, although the models consistently got better results with lower layers amount:

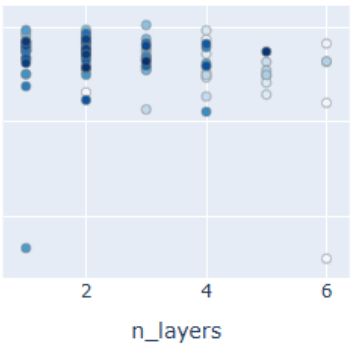


Figure 28 Optuna trials validation accuracy as a function of layers amount

Lagged XGBoost Hyperparameter Tuning

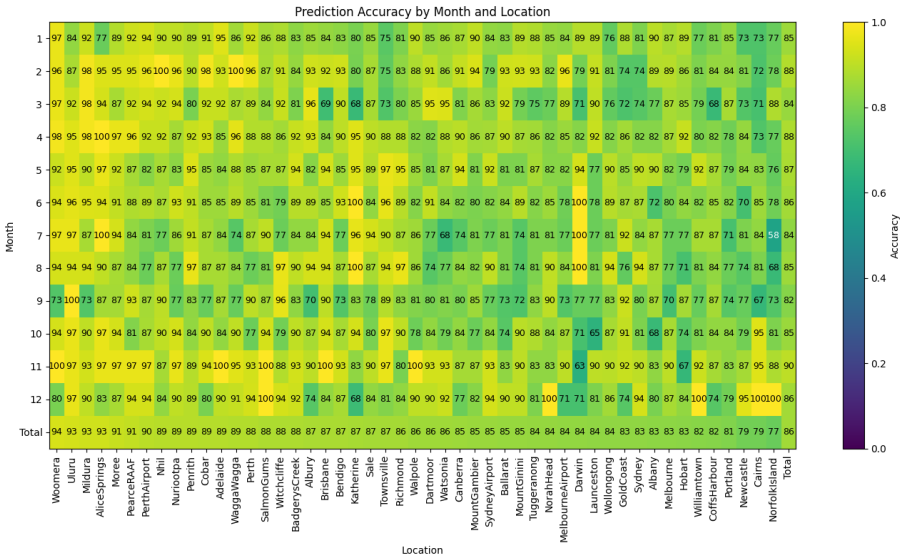
Commented [DN11]: This section wasn't in the presentation.

Again, I wanted to check if I can get better results by providing additional days.

So, I ran 100 Optuna trials with the number of lagged days (K) as a hyperparameter.

My best validation accuracy was an 87.27% and 64% f1 score.

With test accuracy of 86.16% and 65% f1 score.



Interestingly, trials with lower lagged amount did better, although the best trial still better than the best one with “0” lagged days.

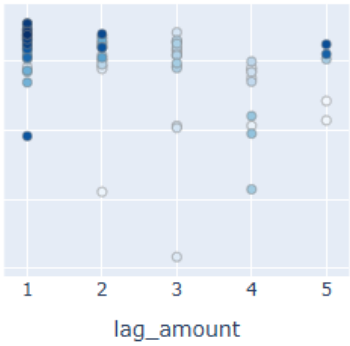


Figure 29 Optuna trials validation accuracy as a function of lag days amount (K)

LSTM Hyperparameter Tuning

Commented [DN12]: This section wasn’t in the presentation.

In a time-series data, like this one, using recurrent neural networks is natural.

As explained before, some features don’t have to be repeated so I defined 2 architectures:

1. A standard LSTM network with those features do repeat.

2. An LSTM network with those features provided separately at the end and aggregated by a simple feed-forward network.

Additionally, the number of days to consider (similarly to “k”) was a hyperparameter as well with a range of 3-10.

I ran 100 Optuna trials each with 30 epochs and chose the epoch with the highest validation accuracy.

My best validation accuracy was 86.75%.

With test accuracy of 85.7%.

There wasn’t a clear best architecture, other than providing the features repeated:

params_bidirectional	params_dropout	params_hidden_dim	params_learning_rate	params_num_layers	params_repeated	params_seq_length
False	0.333613	68	0.000366	1	True	7
True	0.486325	265	0.000143	4	True	5
False	0.485606	511	0.000102	1	True	5
True	0.408109	264	0.000248	4	True	5
False	0.489961	452	0.000177	1	True	5
True	0.184968	39	0.000146	3	True	6
False	0.411971	123	0.000634	2	True	5
True	0.451402	261	0.003046	3	False	10

Figure 30 Top Optuna trials parameters

Note, I think the performance difference (in all the examples above) between the validation and test happens because of 2 main reasons:

1. The climate changes a little through the years, so as years advance the training data becomes outdated. (The climate changes for example because of global warming)
2. I tuned the parameters according to the validation which might create some indirect training.

EDA Research

In this section I try to explain when/why the model struggles to accurately predict the rain.

The researched model in this section is a default XGBoost model that trained on the data after the imputation of sunshine and evaporation.

The model struggles with alternating rainy days

By comparing the results of the XGBoost model to our baseline, we can see that the improvement uniformly, that is, months and locations where the Baseline has a high accuracy, the XGBoost model does good as well. While months and locations where the Baseline has a low accuracy, the XGBoost model struggles as well:

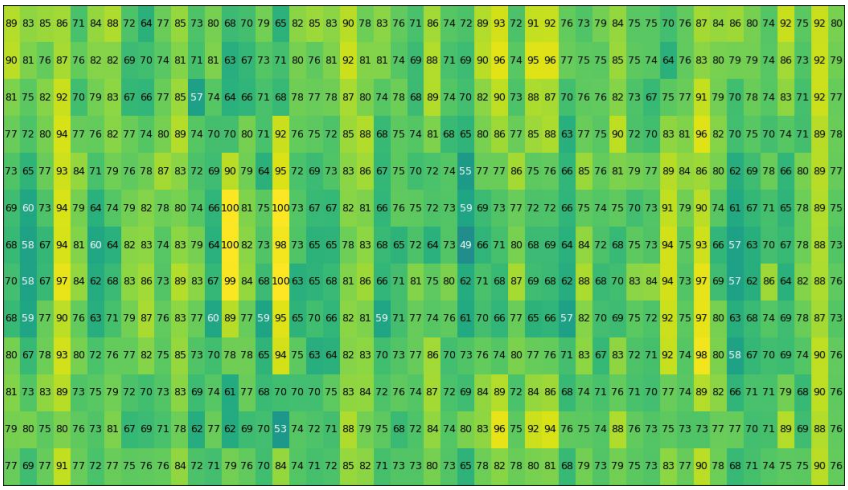


Figure 31 Alternating accuracy of the baseline vs a XGBoost model across months and locations

Let's, for example, look at Watsonia in July (because the model struggles there):

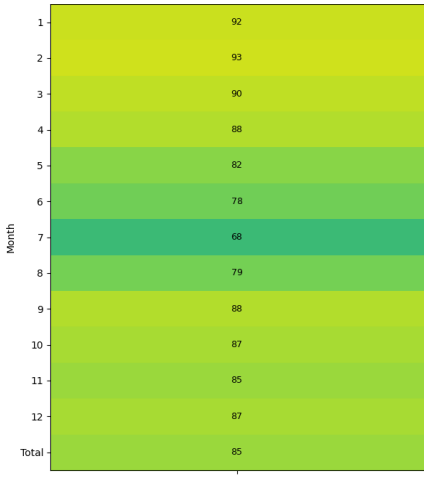


Figure 32 Accuracies across months of an XGBoost model trained on Watsonia

As we can see, the model struggles to predict correctly in a month with high alternating frequency between rainy days and dry days:

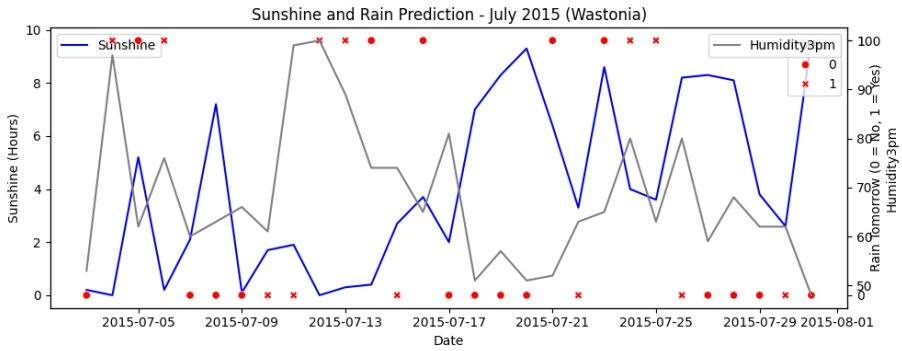


Figure 33 Relatively low accuracy of the model in Watsonia July 2015, showed by providing:
** The 2 most important features (Sunshine in blue and Humidity in gray),*
** what days it rained the following day (red at the bottom indicates no rain tomorrow, while on top indicates it will rain tomorrow),*
** the model prediction as circles and Xs (circle meaning no rain tomorrow while X means it will).*

In contrast to months with constant precipitation:

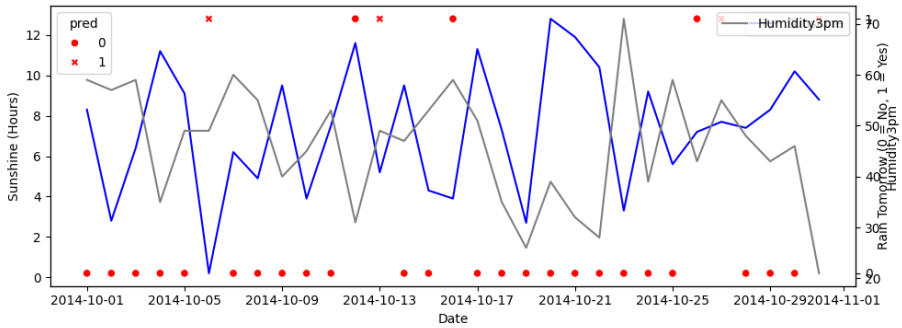


Figure 34 Relatively high accuracy of the model in Watsonia October 2014

Model's OOD challenge

Another reason the model struggles in Watsonia in July is because its weather is out of distribution.

For example, let's look at the sunshine distribution in the training data, validation failing examples and out month – July:

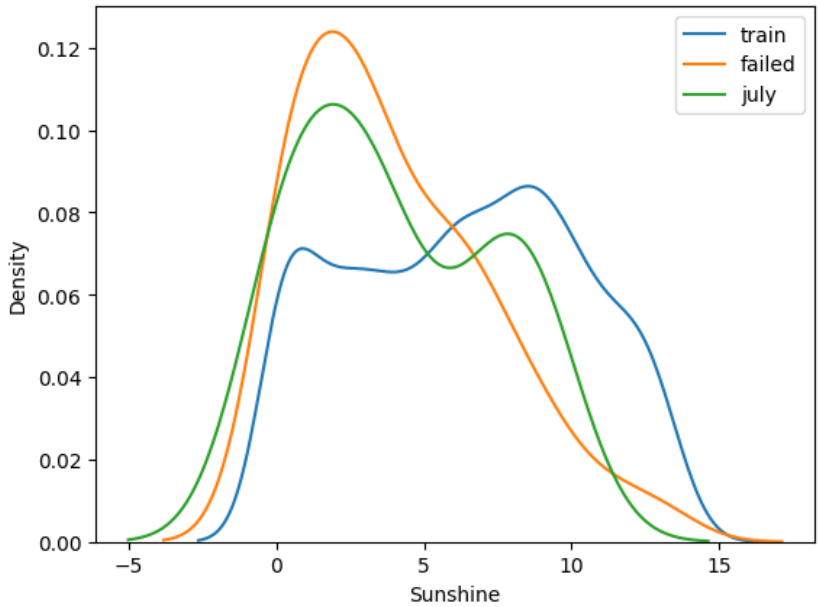
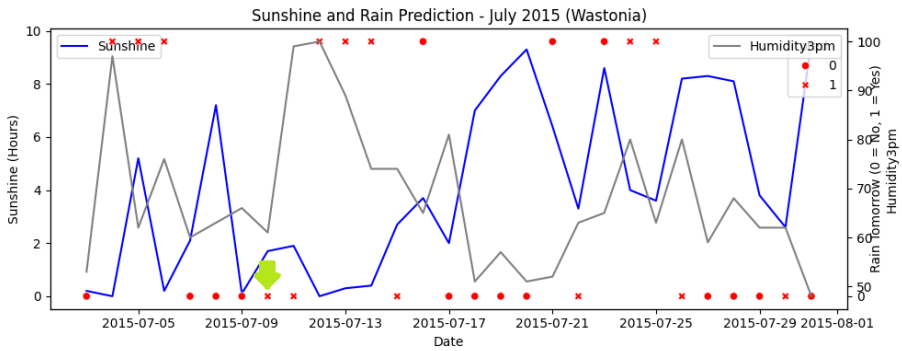


Figure 35 Sunshine distribution in the training data, failed samples in the validation and in July.

As we can see, the Sunshine distribution is different between the training data and the month of July. Moreover, July's sunshine distribution is closer to the failed samples sunshine than to the training data.

Specific day EDA

Here I will try to explain why the model fails on this specific sample (FP):



By looking at the SHAP explanation we can get a glance at why the model expected rain tomorrow:

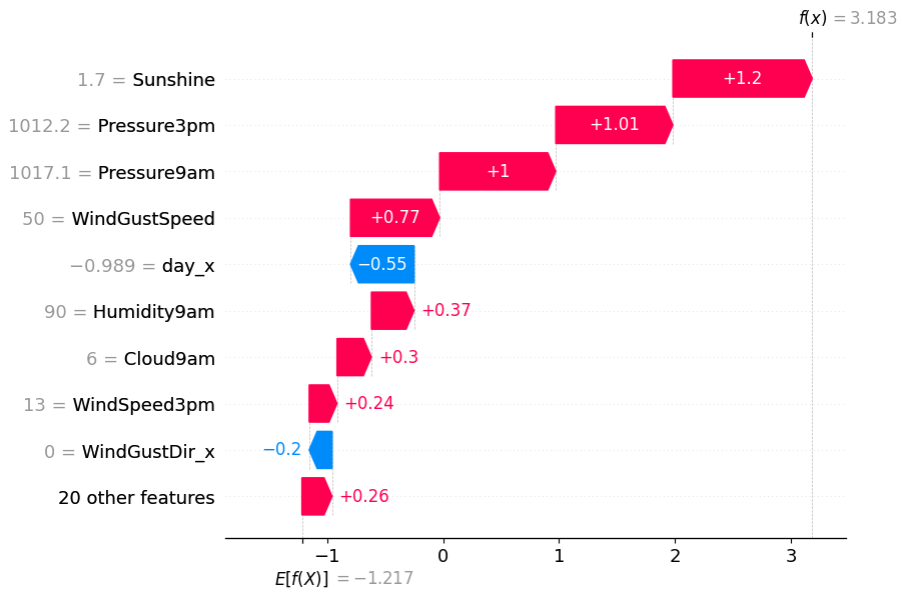


Figure 36 SHAP waterfall explanation on the sample prediction

As we can see, there were several features that pushed the model to predict rain, most importantly, because the Sunshine value was low, the model heavily expected it to rain.

Conclusion

In summary, the primary goal of predicting whether it will rain tomorrow was achieved with 86.16% accuracy and 65% F1-score on the test set, Which significantly outperforms the Baseline of “rain today predicts rain tomorrow”. However, results are not yet optimal, leaving significant room for further improvement.

Regarding the secondary objective, the imputation of sunshine was accurate, especially when incorporating solar exposure data. Although, the evaporation could have been better if provided with more recent data.
Fortunately, the importance of sunshine for rain prediction is significantly higher than evaporation’s importance.